

LogFiT: Log Anomaly Detection using Fine-Tuned Language Models

Crispin Almodovar¹ Fariza Sabrina¹ Sarvnaz Karimi² Salahuddin Azad¹

¹Central Queensland University, Australia

²CSIRO Data61, Sydney, Australia

crispin.almodovar@cquemail.com

{f.sabrina,s.azad}@cqu.edu.au

{sarvnaz.karimi}@csiro.au

Abstract—System logs are a valuable source of information for monitoring and maintaining the security and stability of computer systems. Techniques based on Deep Learning and Natural Language Processing have demonstrated effectiveness in detecting abnormal behavior from these system logs. However existing approaches are inflexible and impractical: techniques that rely on log templates are unable to handle variability in log content, while classification-based approaches require labeled data for supervised training. In this paper, a novel log anomaly detection model named LogFiT is proposed. The LogFiT model is robust to changes in log content and only requires self-supervised training. The LogFiT model uses a pretrained BERT-based language model fine-tuned to recognise the linguistic patterns of the normal log data. The LogFiT model is trained using masked token prediction on the normal log data only. Consequently when presented with the new log data, the model’s top-k token prediction accuracy is used as threshold for determining whether the new log data has deviated from the normal log data. Experimental results show that LogFiT’s F1 score exceeds that of baselines on the HDFS, BGL and Thunderbird datasets. Critically, when variability in the log data is introduced during evaluation, LogFiT’s effectiveness surpasses that of the baseline models.

Index Terms—Service monitoring, fault management, log anomaly detection, deep learning, natural language processing, language modeling.

I. INTRODUCTION

Annually, cyber crime results in billions of dollars of losses for businesses [1]–[3]. The detection of log anomalies can assist in safeguarding the digital infrastructure of businesses from cyber-attacks by identifying abnormal activities, such as network intrusions, from large amounts of event logs created by networked computer systems.

Recent research efforts in log anomaly detection have employed Deep Learning and Natural Language Processing (NLP) techniques to address this issue. A literature review reveals that state-of-the-art research in this area employs the Long Short-Term Memory (LSTM) architecture, as exemplified by the DeepLog model [4], and Transformers, represented by the LogBERT model [5]. One crucial factor in applying deep learning to log anomaly detection is the availability of labeled data for training predictive models.

Due to the high cost of preparing labeled data, classification-based methods such as LogSy [6] have limited utility in production settings. As a result, the majority of techniques used in detecting log anomalies assume a zero-positive training scenario, where normal log data is the only data available for self-supervised training [7] only. Furthermore, [8] considers two categories of self-supervised models for log anomaly detection. The first is forecasting-based models which attempt to predict upcoming log entries by analyzing previous log entries, while the second is reconstruction-based models which reconstruct log sequences that have been intentionally corrupted. The DeepLog model uses the forecasting-based method, whereas the LogBERT model uses the reconstruction-based method.

This study focuses on log data comprising sequences of log sentences that represent various system events, such as the successful connection to an email server or a failure to upload a file after multiple attempts. To train anomaly detection models, log sentences are grouped according to specific criteria, such as session identifier or time window. Each group of log sentences defines a sequence of system events that occurred within a given time window or session. For instance, a normal sequence of log sentences may consist of one or more “File opened successfully” entries followed by “File write operation completed” and/or “File read operation completed” entries. However, a sequence of log sentences that only contains “File opened successfully” entries without corresponding “read” or “write” entries can be considered an anomaly. This information is critical to identifying anomalies in system logs and developing effective anomaly detection models.

The quality of the representation of log sequences is a critical factor that impacts the effectiveness of log anomaly detection models, especially when log content evolution is taken into account [9], [10]. In order to represent log sequences, a common approach is to first convert log sentences into log templates [4], [5]. A set of log templates is thus produced and this set of log templates functions as a form of input vocabulary. Afterwards all input log sentences are mapped to an entry within the set of log templates. This approach has been shown to negatively effect

model effectiveness because of its semantically deficient representations of log sentences. Furthermore the input log sentences are expected to match a log template. Thus this approach is unable to handle variability in the content log sentences over time [6], [11], [12].

To address the limitations of existing approaches for anomaly detection in system logs, this work makes the following contributions:

- A novel log anomaly detection model, named LogFiT, which utilises a pre-trained Bidirectional Encoder Representations from Transformers (BERT) based language model (LM) that has been fine-tuned to understand the linguistic and sequential patterns of the normal log data. The LogFiT model is able to produce representations for log sequences of up to 4096 tokens by leveraging a pre-trained LM. This ensures that the model remains robust even when there are unforeseen changes to the content of log sentences.
- A framework and workflow based on the HuggingFace ecosystem. The framework adopts the masked language modeling (MLM) self-supervised training objective. During training, the model is trained on normal data, with the aim of minimising the cross-entropy loss between predictions and the actual data. At inference time, the framework determines whether a log sequence is normal or an anomaly by comparing the model's top-k accuracy against a pre-defined threshold value.
- We have conducted experiments comparing LogFiT's precision, recall, F1 score and specificity against two baseline approaches, DeepLog and LogBERT. the experiments were conducted using three datasets: HDFS, BGL, and Thunderbird. We present the experimental results to demonstrate the superiority of LogFiT in detecting anomalous log sequences compared to the baseline approaches.
- We will make our implementation of the model, developed using Python, PyTorch and HuggingFace, publicly available, along with the model checkpoints for reproducibility and further research.

II. RELATED WORK

The process of log anomaly detection generally involves four key steps, which have been identified by various researchers [13]–[16].

- **Log data pre-processing** - which involves cleaning and standardising the raw system logs that have been generated by computer systems. This step is critical as the accuracy of the subsequent steps relies on the quality of the pre-processed data. It is assumed that the logs have already been collected at this stage, and the focus is on standardising the data and on removing any inconsistencies and irrelevant information.
- **Vectorisation** – this step is concerned with transforming the pre-processed log data into numeric representations called vectors. The training of machine learning models in the following step requires input data to be in the form of vectors.

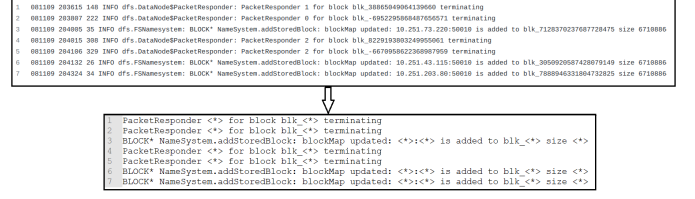


Figure 1. HDFS log sentences converted to log templates.

- **Model development** – during this step the optimal combination of training objective, neural network architecture and performance evaluation metrics is selected. Once these elements are identified, the neural network model is trained on the vectorised data obtained in the preceding step, using an optimisation algorithm to minimise the difference between the model's predictions and the actual output. Finally, the model's performance is evaluated based on the selected metrics, and the best-performing model is selected.
- **Model operationalisation** – in this final step, the best performing machine learning model is operationalised by deploying it to a production environment. The model's efficiency and effectiveness are then monitored continuously to ensure it performs optimally in its given environment.

The initial three activities of log anomaly detection have been the subject of extensive research. For log data pre-processing, some studies have emphasised the use of **log parsing** [4], [5]. However, more recent studies [11], [17] suggested that log parsing may lead to a reduction in accuracy and hence, should not be recommended. To increase accuracy, a state-of-the-art method called **semantic vectorisation** is applied to the log data, to enrich it with semantic information before using them as input for model training [11], [17]. In contrast, the earlier cited works [4], [5] have utilised basic numerical encoding techniques to convert the input log data into vectors. In model development, previous studies have revealed that the best performance in log anomaly detection were attained through the use of model architectures and techniques that were inspired by natural language processing (NLP), as noted in Pang, Shen, Cao, *et al.* [13] and Yadav, Kumar, and Dhavale [18]. Among these techniques, Long Short-Term Memory (LSTM) and Transformers have been the preferred model architectures [4]–[6], [11].

A. Log Templates

Log anomaly detection methods commonly use log parsing as the initial step, which involves converting log data into a standardised format known as **log templates** [14], [16], [19]. The goal of log parsing is to map each log sentence to a specific log template, which forms the vocabulary of the model, rather than using words or tokens typical in **natural language processing** (NLP). Several log anomaly detection methods, such as DeepLog [4], LogRobust [17], and LogBERT

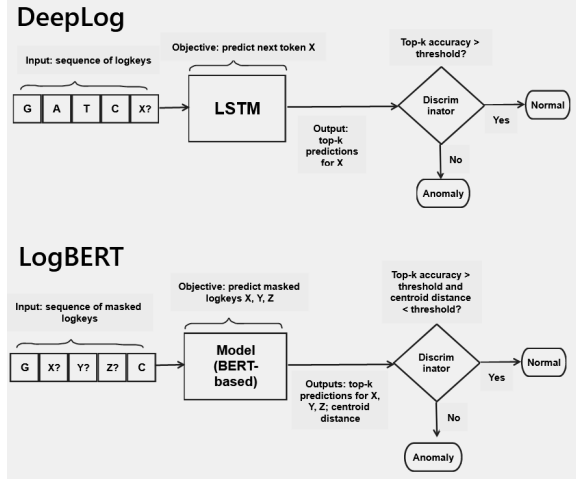


Figure 2. Comparison of the DeepLog and LogBERT log anomaly detection methods.

[5], rely on log templates, often generated using the Drain parser [20], to pre-process log data.

Figure 1 illustrates how log sentences are converted into log templates in the DeepLog and LogBERT methods. The conversion of log data to log templates is done as part of the pre-processing step, and the generated set of log templates represents the model vocabulary. As illustrated in Figure 2, the input to the DeepLog and LogBERT models are a sequence of **log keys**, where each log key corresponds to a log template stored in the vocabulary. In the DeepLog method, the last log key is masked in the input and the model is tasked with predicting what the masked log key was, given the log keys that precede it. In contrast, LogBERT masks out random log keys in the input sequence and tasks the model with predicting what all the masked tokens were.

Several recent studies, conducted by Nedelkoski, Bogatinovski, Acker, *et al.* [6], Le and Zhang [11], and Wittkopp, Acker, Nedelkoski, *et al.* [12], have highlighted that the use of log templates can result in a loss of contextual information, which can adversely affect the accuracy of predictive models. Moreover, models developed using log templates can become dependent on the log parsing tool employed. Approaches that rely on log templates assume that the set of log templates remains fixed over time, but in reality, changes in the content of log sentences can occur naturally. As a result, models that depend on log templates may not be able to match unseen log sentences to an existing record in the set of log templates.

These issues were addressed in the development of several log anomaly detection models, including LogSy [6], Neuralog [11], and A2Log [12]. Rather than relying on log parsing to generate log templates, these models pre-process log data using plain cleanup scripts to eliminate extraneous details such as particular IP addresses, file paths, port numbers, and internet URLs.

B. Semantic Vectors

The literature on log anomaly detection reveals a growing trend towards the adoption of semantic vectorisation

techniques, which enable models to encode the linguistic structure and meaning of log data. The LogRobust model [17] utilised static word embeddings to incorporate semantic information to log data representations. The proponents of the LogRobust model observed that word embeddings increased the model’s accuracy.

C. Pre-trained Language Models

In recent research, there has been a growing interest in the use of pre-trained **language models** (LMs) such as BERT [21] to improve anomaly detection in system logs. Studies by Ott, Bogatinovski, Acker, *et al.* [10] and Le and Zhang [11] demonstrated that pre-trained LMs can offer significant advantages over word embeddings, which were used in the LogRobust model [17]. According to these studies, pre-trained LMs capture contextual information at the level of the **whole sequence** of log sentences, whereas word embeddings only provide representations for individual words in a single sentence. Furthermore, BERT generates context-sensitive semantic vectors that encode the **order** of words within a log sentence and are capable of handling **out-of-vocabulary** words. In contrast, the semantic vectors generated via word embeddings are static and are computed by computing the mean of the word embeddings of all the words comprising a log sentence.

Existing research suggest that pre-trained LMs such as BERT can learn and incorporate both syntactic and semantic information that can improve the effectiveness of natural language processing tasks [22]–[25]. Therefore, in this study, a pre-trained LM is incorporated in the LogFiT model to leverage its ability to understand the sequential properties and linguistic structure of system logs.

III. LOGFiT

The proposed LogFiT approach takes advantage of recent advances in Deep Learning and NLP to detect anomalies in system event logs. It leverages the linguistic capabilities of what Stanford University’s Institute for Human-Centered Artificial Intelligence refer to as “foundation models” [26], which are models that have been pre-trained on very large, multi-modal corpora (text, images, video, etc.), and are intended to be used as a foundation (via transfer learning) for downstream NLP, image, and video tasks. Specifically, LogFiT uses the Longformer [27] model as its base. The LogFiT model is essentially a pre-trained Longformer that has been fine-tuned to learn the linguistic structures and sequential properties of log data. The Longformer model is an enhanced variant of the RoBERTa model [28] which in turn is an improvement over the original BERT model [21]. The Longformer model is chosen because of its ability to support more than 512 tokens - which is the limit for BERT.

LogFiT adopts a self-supervised training approach whereby the model is trained on the normal log data so that it only learns the linguistic patterns of the normal log data. The training objective of the LogFiT model is to predict a number of masked tokens in the log sentences, so it makes use of cross-entropy loss to minimise the difference (i.e. “loss”) between

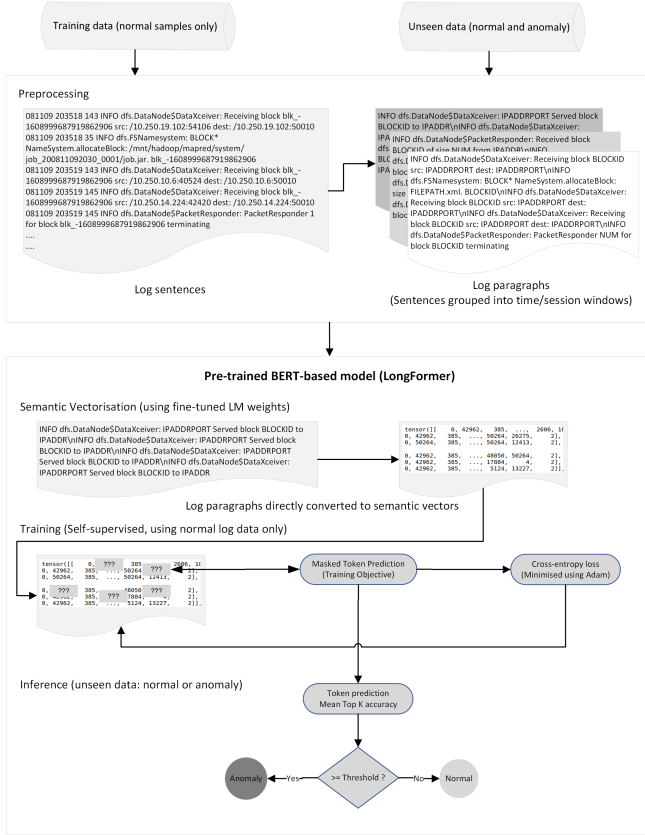


Figure 3. Logical architecture of the LogFiT log anomaly detection approach.

the model’s predicted tokens and the real tokens. Because the cross-entropy loss is logarithmic, it yields larger values for incorrect predictions than for correct predictions. Further, the model computes semantic vectors representing the log paragraphs being processed; the contextual embedding vector for the [CLS] token is used as the semantic vector. The [CLS] embedding vectors are computed for downstream tasks such as clustering and visualisation. Figure 3 illustrates the LogFiT approach and its model training and inference workflow. An early version of the LogFiT method is described in Almodovar, Sabrina, Karimi, *et al.* [29].

A. Framework

Input Representation. The LogFiT model utilises normal log data for training, which has been converted to semantic vectors prior to being passed to the model. In contrast to both the DeepLog and LogBERT methods, the LogFiT model does not require the input log data to first be converted to log templates. Rather, log data is directly processed, tokenised, and transformed into semantic vectors in a single step. LogFiT takes inspiration from recent models [10], [11] that forego the log template extraction step, instead directly converting the log data into semantic vectors using a pre-trained LM. However, neither of the two aforementioned models fine-tune the pre-trained LM, instead utilizing the LM as a static semantic vectoriser.

Transformer architecture. LogFiT inherits from the innovations introduced by the BERT-based language models.

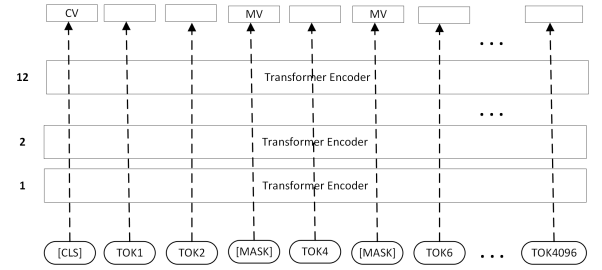


Figure 4. The LogFiT model’s Transformer encoder layers, showing token embeddings are computed from the input tokens.

The BERT model is a Transformer encoder model that has the capabilities of an auto-encoder. The BERT model’s encoder capability allows it to generate semantic vectors that are sensitive to the full context of the input log paragraph, while enabling it to reconstruct log paragraphs that have been corrupted [21]. BERT’s ability to accurately reconstruct corrupted input logs can be used as a threshold-based anomaly detection method. Thus by inheriting from BERT, LogFiT includes both the vectoriser and log anomaly detection component in a single architecture, allowing for end-to-end training that does not require intermediate log template extraction or vectorisation steps. As mentioned earlier, LogFiT uses the Longformer [27] model, which allows LogFiT to process log paragraphs containing up to 4096 tokens, which is longer than the 512-token limit of the BERT model. The Longformer model overcomes BERT’s limitation on the number of tokens, due to the quadratic computational complexity of BERT’s attention mechanism, by introducing a sliding window strategy which effectively reduces attention computation to a linear time [27].

The LogFiT model consists of 12 stacked Transformer encoders, 12 attention heads per layer, 768-dimension vectors, a maximum possible sequence length of 4096 tokens – as illustrated in Figure 4. Not shown is the embedding layer, which is typically implemented inside the first Transformer encoder.

B. Training Objective

As mentioned previously, the LogFiT model is trained in a self-supervised manner using *masked token prediction*. This training objective is modified version of the self-supervised *masked language modeling* (MLM) training objective used to pre-train BERT-based language models [21]. The LogFiT model randomly chooses up to 50% of the sentences that constitute the log paragraph for masking, instead of random tokens from all the sentences of the log paragraphs as is the case with BERT. Next, the chosen log sentence’s tokens are masked following the BERT masking algorithm, with 80% of the tokens being masked while the rest are left unmodified. Afterward, the model is tasked with predicting what the masked tokens were.

The rationale for using masked token prediction is that the model needs to learn the contextual relationships between tokens and sentences in the training data, in order to accurately

predict the masked tokens. This enables the model to develop an understanding of the language rules used by normal system logs. As a result, it can differentiate normal log data from anomaly log data.

To implement the masked token prediction training objective, the model minimises the cross-entropy loss between its masked token predictions and the actual tokens. The computation of the cross-entropy loss for a mini-batch of log data is shown in Equation 1. The LogFiT model minimises the training loss using the Adam optimiser, with hyper parameters values adopted from FastAI defaults: *momentum* = 0.9, *sqr_momentum* = 0.99, *epsilon* = 1e-5, and *weight_decay* = 0.01.

$$Loss = -\frac{1}{b} \sum_{j=1}^b \sum_{i=1}^m y_{mask_i}^j \log(p_{mask_i}^j) \quad (1)$$

where b is size of the the mini-batch, m is the number of masked tokens, y and p are the true and predicted values, respectively.

To speed up the model training, LogFiT adopts super-convergence techniques introduced by Smith and Topin [30] and implemented in the FastAI/ULMFiT framework [31]. The super-convergence techniques are: (i) *discriminative fine-tuning*, in which different learning rates are used for different layer groups of the model, (ii) *slanted triangular learning rates and one-cycle learning rates*, in which the learning rate and momentum are optimally set during training, and (iii) *gradual unfreezing*, in which the model's weights are trained one layer group at a time. While ULMFiT is primarily for LSTM architectures, the techniques that it embodies have been validated to be effective for Transformer-based architectures as well [32], [33]. Gururangan, Marasović, Swayamdipta, *et al.* [32] have shown that task-adaptive pre-training (i.e. "fine-tuning") lead to significant (2% on average) performance gains, while Kumar, Raghunathan, Jones, *et al.* [33] showed that the "gradual unfreezing" approach mitigates the observed phenomenon of reduced effectiveness when LMs are fine-tuned, due to the dissipation of the pretrained weights.

C. Anomaly Detection

The LogFiT model, which is exclusively trained on normal data, can then be used to detect abnormal log data. During the inference stage, log paragraphs are tokenised, vectorised, masked, and reconstructed in the same way as during training. To determine whether a log paragraph is anomalous, LogFiT uses a technique adopted from DeepLog and LogBERT, where the top- k accuracy (with k ranging from 5 to 15) is used as an anomaly score. The accuracy score indicates how effectively LogFiT can restore the masked sentences in the input data. If the model's top- k accuracy falls below a certain threshold (ranging from 0.8 to 0.99 in the experiments), the log paragraph is classified as anomalous, while a score above the threshold indicates that it is normal.

IV. EXPERIMENTS

In this section the datasets, experimental setup and implementation details are described. Subsequently, the results of running the experiments are evaluated.

A. Experimental Setup

Datasets. The LogFiT model is trained and evaluated using three public datasets: HDFS, BGL and Thunderbird. These are the same datasets used by the baseline models that LogFiT is compared against. It is important to note that while these datasets are labeled, LogFiT only uses the labels to validate the performance of the model during training and evaluation. It is expected that in typical application scenarios, log data are not labeled. Moreover, when LogFiT is deployed, it is designed to operate in online mode rather than batch mode. Consequently, for datasets where the grouping of log sentences is based on time windows (such as BGL and Thunderbird), a time interval of 30 seconds is chosen to ensure that a system that employs LogFiT can deliver prompt feedback to system operators. Table I shows some statistics about the HDFS, BGL and Thunderbird datasets.

- **HDFS system logs.** These logs were generated by an installation of Hadoop Distributed File System (HDFS) [34]. The log dataset contains both normal log events and anomaly log events, which have been manually tagged by domain experts. In the HDFS dataset, anomaly system events represent abnormal file system operations as specified by the flow of blocks across the HDFS cluster. Example types of anomalies are "Replica immediately deleted", and "Namenode not updated after deleting block". The HDFS dataset is chunked into log paragraphs using the HDFS block ID, which represents a session in HDFS. The full HDFS dataset consists of 11,175,629 log sentences, of which 16,838 are anomalies.
- **BGL system logs.** These logs were produced by the Blue Gene/L supercomputer system installed at the Lawrence Livermore National Laboratory (LLNL) [35]. The dataset contains suitably labeled normal and alert events, with the alert events considered as anomalies. The BGL dataset does not have a session ID with which to group the data, thus the dataset is chunked into log paragraphs, each consisting of the consecutive log sentences belonging to a time window of 30 seconds, following the approach discussed in [5], [7]. A log paragraph is considered an anomaly if it includes at least one log sentence that is tagged as an anomalous event. The full BGL dataset consists of 4,747,963 log sentences, of which 348,460 are anomalies.
- **Thunderbird system logs.** These logs were created by the Thunderbird supercomputer system operated by Sandia National Laboratories (SNL) [35]. Similar to the BGL system logs, the Thunderbird system logs contain data about the activities and events that occurred within the supercomputer system. The dataset has also been labeled in the same manner as the BGL data set. As with the BGL dataset, the Thunderbird dataset does not have a session ID, so the log sentences are

Dataset	Avg Word Count	Avg Sentence Count	#Unique Words
HDFS	176.04	18.63	146
BGL	128.66	15.73	6046
Thunderbird	1445.70	126.63	15557

Table I

PER-PARAGRAPH WORD AND SENTENCE STATISTICS FOR THE DATASETS.

similarly chunked into paragraphs, each consisting of the consecutive sentences belonging to a time window of 30 seconds, following the approach discussed in [5], [7]. Furthermore, a log paragraph is considered an anomaly if it includes at least one log sentence that is tagged as an anomalous event. The full Thunderbird dataset consists of 10,000,000 log sentences, of which 4,934 are anomalies.

For each of the datasets mentioned, a total of 5,000 log paragraphs were used to train LogFiT (sampled from 10,000 log paragraphs set aside for training). A separate tuning set consisting of 4,500-5,000 normal plus 500-1,000 anomaly paragraphs were used to tune the LogFiT model’s hyper parameters. Furthermore, a separate evaluation set consisting of 4,500-5,000 normal and 500-1,000 anomaly samples was set aside to evaluate the performance of the tuned LogFiT model. No random shuffling is done when splitting the datasets into subsets - the sequential order of the logs is preserved; this is to prevent information leakage during model training. Lastly, to test the models’ ability to handle variation in the syntactic structure of the input log data, the evaluation set is dynamically modified during model evaluation, so that the top 10% most commonly occurring verbs are replaced with their WordNet lemmas.

Baselines. The performance of the LogFiT model is compared against the following baselines.

- **DeepLog [4].** The DeepLog model uses an LSTM-based architecture to train an anomaly detection model using sequences of normal log templates. Anomalies are detected by letting the model predict the next log template given its n preceding log templates. The top- k accuracy of the prediction is the anomaly score. The DeepLog implementation from the logdeep library ¹ was used to obtain the results reported in the experiments section. It is worth noting that the performance metrics reported in the DeepLog paper cannot be reproduced using this implementation of DeepLog.
- **LogBERT [5].** The LogBERT model uses a BERT-based architecture to learn the patterns of normal log templates, using masked log key prediction and centroid distance minimisation to fit the model to the training data. Anomalies are detected by allowing the model to predict n randomly masked log keys in a sequence, and then using the top- k accuracy and centroid distance to compute an anomaly score. If either the top- k accuracy or centroid distance threshold is exceeded, the input data is considered an anomaly. The publicly available LogBERT source code ² is used to obtain the results reported in

the paper. It is worth noting that the performance metrics reported in the LogBERT paper cannot be reproduced.

Implementation Details. LogFiT was implemented using Python and leveraged several well-known libraries to accelerate the development and evaluation of the model.

- Pre-processing of the raw log data was done using simple regular expressions to replace unimportant (because they are too specific) details such as IP addresses and port numbers, numeric values, file paths, block IDs (and other types of identifiers). Python and its extensive data analysis tools were used for this task.
- Pytorch - Pytorch is a Python-based deep learning framework developed by Facebook Research. Pytorch allows researchers to build and train complex neural network models using a Pythonic syntax that is intuitive and easy to use. LogFiT uses Pytorch as interface to the underlying hardware-accelerated tensor runtime.
- FastAI - FastAI is a high-level deep learning framework that has been designed to run on top of Pytorch. This framework focuses on efficient model training workflows and incorporates the latest deep learning techniques and best practices. LogFiT uses FastAI to implement its training and evaluation strategies in concise Python code.
- HuggingFace - HuggingFace is a Python library and ecosystem for building and sharing Transformers-based models. LogFiT uses the HuggingFace library to jump start model development by inheriting from pre-trained Transformer models available from the HuggingFace model hub.

The source code implementing the LogFiT model, datasets and model checkpoints will be made available online.

Evaluation Metrics. To evaluate the effectiveness of the models, the experiments use the following metrics:

- *Precision (P)* measures the proportion of correctly identified anomaly samples (TP), out of all the anomalies detected by the model, and is calculated as $P = TP / (TP + FP)$
- *Recall (R)* measures the proportion of correctly identified anomaly samples (TP) out of all real anomalies, and is calculated as $R = TP / (TP + FN)$.
- *F1 Score ($F1$)* is the harmonic mean of the Precision and Recall, and is calculated as $F1 = 2 * (P * R) / (P + R)$.
- *Specificity (S)* measures the proportion of correctly identified normal samples (TN) out of all real normal samples, and is calculated as $S = TN / (TN + FP)$.

True Positives (TP) refer to the number of anomaly samples that were correctly detected by the model. **False Positives (FP)** refer to the number of normal log samples that the model incorrectly detected as anomaly. **False Negatives (FN)** are the anomaly samples that were not detected by the model. Finally **True Negatives (TN)** are the normal samples that the model correctly detected.

In real-world deployment scenarios, having a predictive model with high Specificity is more advantageous since it minimises the chances of producing false positives or false alarms. A model with high Specificity can correctly identify normal samples, which makes it more likely that any detected

¹ Available from <https://github.com/donglee-afar/logdeep>

² Available from <https://github.com/HelenGuohx/logbert>

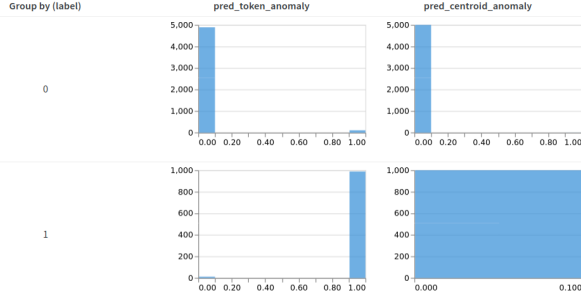


Figure 6. Aggregate anomaly predictions for the Thunderbird evaluation dataset, with 5,000 normal and 1,000 anomaly samples. The predictions based on top- k accuracy are more effective than predictions based on centroid distance. The centroid distance threshold simply classifies all samples as normal.

1,000 anomaly samples. The aggregate results for the normal samples are represented by the row/column labeled 0, while the aggregate results for anomaly samples are represented by the row/column labeled 1. The column labeled `pred_token_anomaly` represents the LogFiT predictions (based on top- k prediction accuracy) for the 5,000 normal samples. The column labeled `pred_centroid_anomaly` represents the model’s predictions (based on centroid distance) for the 1,000 anomaly samples. Figure 6 shows how the two anomaly thresholds top- k prediction accuracy and centroid distance, influence the model’s anomaly decision on the Thunderbird dataset. The figure shows that centroid distance is not an effective method for distinguishing normal and anomalous log paragraphs. In this case, all samples were categorised as normal by the centroid distance criteria.

To investigate the reason why centroid distance is not effective, the semantic vectors of the training and evaluation sets are plotted after reducing the vector dimensions to 2 using the UMAP algorithm [36]. As shown in Figure 7, the semantic vectors of the training set do not form clean clusters. So there are no clusters that can effectively threshold the semantic vectors computed during inference. Therefore, it can be concluded that centroid distance minimisation does not improve the performance of LogFiT and can be omitted from the model. In the above mentioned figures, the color of the points represent training samples (blue), normal predictions (pink) and anomaly predictions (yellow). It can be seen that the clusters for the anomaly samples from the evaluation set (yellow points) appear very close to the training/normal clusters (blue points). In contrast the clusters for the normal samples from the evaluation set (pink points) appear further and more spread out from the training/normal clusters (blue points). Therefore centroid distance cannot be used to separate normal from anomaly log paragraphs.

b) Variations in input data: In practical applications, it is expected that the content of the log sentences will change over time. This can be because the programmers may change some words in the log sentences, or introduce misspellings. The LogFiT model contains built-in support for log sentence variability due to its large vocabulary of sub-word tokens (around 50K). In contrast the DeepLog and LogBERT models would fail if they encounter variation in log sentences that

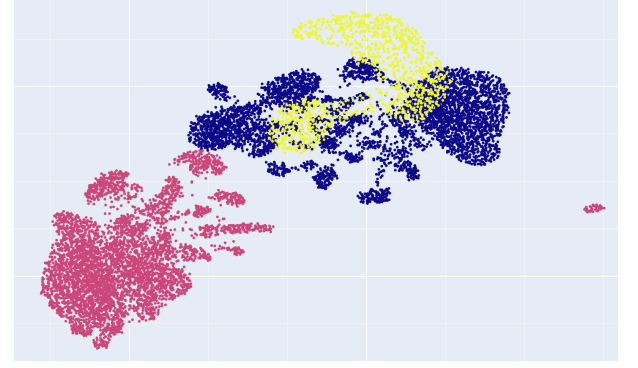


Figure 7. UMAP plot of Thunderbird semantic vectors, where the blue, pink and yellow colours of the points represent training samples, normal predictions and anomaly predictions respectively.

Method	P	R	F1	S
LogBERT	28.82	94.92	44.22	37.18
LogFiT (ours)	95.249	84.2	89.38	99.16

Table V

ANOMALY DETECTION PRECISION (P), RECALL (R), F1 SCORE (F) AND SPECIFICITY (S) OF THE LOGFiT AND LOGBERT METHODS ON THE BGL DATASET. THE EVALUATION DATA WAS MODIFIED TO REPLACE THE TOP 10 VERBS WITH THEIR WORDNET LEMMAS.

cannot be mapped to its list of known log templates. To test the LogFiT model’s ability to handle log sentence variability, the evaluation set is dynamically modified during inference so that the top 10 occurring action words (that can be mapped to synonyms in WordNet) are replaced with their WordNet lemmas. Table V shows the results of feeding the modified BGL evaluation set to the unmodified LogFiT models. The results indicate that the LogFiT model is robust to changes in the log sentences, as the reduction in F1 is around 2% (from 91.22 to 89.38). However the drop in F1 performance for LogBERT is large, from 88.63 to 44.22.

V. CONCLUSION

The paper has introduced LogFiT, a novel log anomaly detection model that leverages the general linguistic knowledge embedded within a pre-trained BERT-based LM and fine-tunes it to recognise specific linguistic patterns of system logs. LogFiT is trained in a self-supervised manner, using only normal system logs, to predict masked tokens in log sequences. This approach enables LogFiT to recognise only the linguistic structure of normal system logs only, and flag anomalies when it fails to reconstruct an input log sequence. Critically, LogFiT is able to handle variability in the content of system logs because of its use of a BERT-based LM. The performance of LogFiT on the HDFS, BGL, and Thunderbird datasets has been evaluated and it has been that LogFiT’s F1 score outperformed that of the baseline models. Moreover, LogFiT’s specificity exceeded that of the baselines on the HDFS and BGL datasets and comparable to LogBERT on the Thunderbird dataset. In addition, LogFiT demonstrated superior performance over LogBERT in experiments that tested for variations in the content of input log paragraphs, which is attributed to its ability to handle out-of-vocabulary

tokens. LogFiT integrates with the popular HuggingFace ecosystem, making it easy to adapt in future work. Overall, LogFiT presents a flexible and future-proof approach to detect abnormal behavior in computer systems through language model adaptation and fine-tuning.

A. Future Work

While the LogFiT model is intended to be used as a threshold-based anomaly detector trained in a self-supervised manner, it can easily be converted to a classifier. If at some point after the model is deployed, operators are able to collect and label anomaly log samples, the model can be converted to a classifier by replacing its language modeling head with a classification head. Additionally, the LogFiT LM can be pretrained on diverse log datasets, allowing it to be used as foundation for downstream NLP and log anomaly detection tasks.

REFERENCES

- [1] RiskIQ, *The evil internet minute 2019*, 2019. [Online]. Available: <https://www.riskiq.com/resources/infographic/evil-internet-minute-2019>.
- [2] Australia Department of Home Affairs, *Australia's cyber security strategy 2020*, 2020. [Online]. Available: <https://www.homeaffairs.gov.au/cyber-security-subsite/files/cyber-security-strategy-2020.pdf>.
- [3] International Business Machines, *Cost of a data breach report 2022*, 2022. [Online]. Available: <https://www.ibm.com/security/data-breach>.
- [4] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning", in *Proceedings of the ACM Conference on Computer and Communications Security*, New York, NY, USA: ACM, 2017, pp. 1285–1298, ISBN: 9781450349468. DOI: 10.1145/3133956.3134015.
- [5] Haixuan Guo, Shuhan Yuan, and Xintao Wu, "LogBERT: Log Anomaly Detection via BERT", *Proceedings of the International Joint Conference on Neural Networks*, vol. 2021-July, Mar. 2021. DOI: 10.1109/IJCNN52387.2021.9534113. arXiv: 2103.04475. [Online]. Available: <https://arxiv.org/abs/2103.04475v1>.
- [6] Sasho Nedelkoski, Jasmin Bogatinovski, Alexander Acker, Jorge Cardoso, and Odej Kao, "Self-attentive classification-based anomaly detection in unstructured logs", in *Proceedings - IEEE International Conference on Data Mining, ICDM*, vol. 2020-Novem, Institute of Electrical and Electronics Engineers Inc., Nov. 2020, pp. 1196–1201, ISBN: 9781728183169. DOI: 10.1109/ICDM50108.2020.00148. arXiv: 2008.09340.
- [7] Van Hoang Le and Hongyu Zhang, *Log-based Anomaly Detection with Deep Learning: How Far Are We?*, 1. Association for Computing Machinery, 2022, vol. 1, ISBN: 9781450392211. DOI: 10.1145/3510003.3510155. arXiv: 2202.04301. [Online]. Available: <http://arxiv.org/abs/2202.04301>.
- [8] Lun Pin Yuan, Peng Liu, and Sencun Zhu, "Recompose Event Sequences vs. Predict Next Events: A Novel Anomaly Detection Approach for Discrete Event Logs", in *ASIA CCS 2021 - Proc. 2021 ACM Asia Conf. Comput. Commun. Secur.*, vol. 1, Association for Computing Machinery, 2021, pp. 336–348, ISBN: 9781450382878. DOI: 10.1145/3433210.3453098.
- [9] Dan Hendrycks, Xiaoyuan Liu, Eric Wallace, Adam Dziedziec, Rishabh Krishnan, and Dawn Song, "Pretrained Transformers Improve Out-of-Distribution Robustness", Association for Computational Linguistics (ACL), Apr. 2020, pp. 2744–2751. DOI: 10.18653/v1/2020.acl-main.244. arXiv: 2004.06100. [Online]. Available: <https://arxiv.org/abs/2004.06100v2>.
- [10] Harold Ott, Jasmin Bogatinovski, Alexander Acker, Sasho Nedelkoski, and Odej Kao, "Robust and transferable anomaly detection in log data using pre-trained language models", in *2021 IEEE/ACM International Workshop on Cloud Intelligence (CloudIntelligence)*, IEEE, 2021, pp. 19–24.
- [11] Van-Hoang Le and Hongyu Zhang, "Log-based anomaly detection without log parsing", in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, IEEE, 2021, pp. 492–504.
- [12] Thorsten Wittkopp, Alexander Acker, Sasho Nedelkoski, *et al.*, "A2Log: Attentive Augmented Log Anomaly Detection", *HICSS 2022 : Hawaii International Conference on System Sciences*, Sep. 2021. arXiv: 2109.09537. [Online]. Available: <http://arxiv.org/abs/2109.09537>.
- [13] Guansong Pang, Chunhua Shen, Longbing Cao, and Anton Van Den Hengel, "Deep learning for anomaly detection: A review", *ACM Comput. Surv.*, vol. 54, no. 2, Mar. 2021, ISSN: 0360-0300. DOI: 10.1145/3439950. [Online]. Available: <https://doi.org/10.1145/3439950>.
- [14] Shilin He, Pinjia He, Zhuangbin Chen, Tianyi Yang, Yuxin Su, and Michael R Lyu, "A survey on automated log analysis for reliability engineering", *ACM computing surveys (CSUR)*, vol. 54, no. 6, pp. 1–37, 2021.
- [15] Raghavendra Chalapathy and Sanjay Chawla, "Deep Learning for Anomaly Detection: A Survey", 2019. arXiv: 1901.03407. [Online]. Available: <http://arxiv.org/abs/1901.03407>.
- [16] Nengwen Zhao, Honglin Wang, Zeyan Li, *et al.*, "An empirical investigation of practical log anomaly detection for online service systems", in *ESEC/FSE 2021 - Proceedings of the 29th ACM Joint Meeting European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, vol. 21, 2021, pp. 1404–1415, ISBN: 9781450385626. DOI: 10.1145/3468264.3473933. [Online]. Available: <https://doi.org/10.1145/3468264.3473933>.
- [17] Xu Zhang, Yong Xu, Qingwei Lin, *et al.*, "Robust log-based anomaly detection on unstable log data", in *ESEC/FSE 2019 - Proceedings of the 2019 27th*

- ACM Joint Meeting European Software Engineering Conference and Symposium on the Foundations of Software Engineering, vol. 19, 2019, pp. 807–817, ISBN: 9781450355728. DOI: 10.1145/3338906.3338931. [Online]. Available: <https://doi.org/10.1145/3338906.3338931>.
- [18] Rakesh Bahadur Yadav, P. Santosh Kumar, and Sunita Vikrant Dhavale, “A Survey on Log Anomaly Detection using Deep Learning”, in *ICRITO 2020 - IEEE 8th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions)*, Institute of Electrical and Electronics Engineers Inc., Jun. 2020, pp. 1215–1220, ISBN: 9781728170169. DOI: 10.1109/ICRITO48877.2020.9197818.
- [19] Zhuangbin Chen, Jinyang Liu, Wenwei Gu, Yuxin Su, and Michael R. Lyu, “Experience Report: Deep Learning-based System Log Analysis for Anomaly Detection”, Jul. 2021. arXiv: 2107.05908. [Online]. Available: <http://arxiv.org/abs/2107.05908>.
- [20] Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R. Lyu, “Drain: An Online Log Parsing Approach with Fixed Depth Tree”, in *Proceedings - 2017 IEEE 24th International Conference on Web Services, ICWS 2017*, 2017, pp. 33–40, ISBN: 9781538607527. DOI: 10.1109/ICWS.2017.13.
- [21] Jacob Devlin, Ming Wei Chang, Kenton Lee, and Kristina Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding”, in *NAACL HLT 2019 - 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies - Proceedings of the Conference*, vol. 1, Association for Computational Linguistics (ACL), Oct. 2019, pp. 4171–4186, ISBN: 9781950737130. arXiv: 1810.04805. [Online]. Available: <https://arxiv.org/abs/1810.04805v2>.
- [22] Ganesh Jawahar, Benoît Sagot, and Djamé Seddah, “What does BERT learn about the structure of language?”, in *ACL 2019 - 57th Annu. Meet. Assoc. Comput. Linguist. Proc. Conf.*, Association for Computational Linguistics (ACL), 2020, pp. 3651–3657, ISBN: 9781950737482. DOI: 10.18653/v1/p19-1356. [Online]. Available: <https://aclanthology.org/P19-1356>.
- [23] Yongjie Lin, Yi Chern Tan, and Robert Frank, “Open Sesame: Getting inside BERT’s Linguistic Knowledge”, 2019, pp. 241–253. DOI: 10.18653/v1/w19-4825. arXiv: 1906.01698. [Online]. Available: <https://github.com/>.
- [24] Yoav Goldberg, “Assessing BERT’s Syntactic Abilities”, 2019. arXiv: 1901.05287. [Online]. Available: <http://arxiv.org/abs/1901.05287>.
- [25] David Yenicecik, Florian Schmidt, and Yannic Kilcher, “How does BERT capture semantics? A closer look at polysemous words”, Association for Computational Linguistics (ACL), Nov. 2020, pp. 156–162. DOI: 10.18653/v1/2020.blackboxnlp-1.15. [Online]. Available: <https://aclanthology.org/2020.blackboxnlp-1.15>.
- [26] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, et al., “On the Opportunities and Risks of Foundation Models”, Aug. 2021. arXiv: 2108.07258. [Online]. Available: <http://arxiv.org/abs/2108.07258>.
- [27] Iz Beltagy, Matthew E. Peters, and Arman Cohan, “Longformer: The Long-Document Transformer”, 2020. arXiv: 2004.05150. [Online]. Available: <http://arxiv.org/abs/2004.05150>.
- [28] Yinhan Liu, Myle Ott, Naman Goyal, et al., “RoBERTa: A Robustly Optimized BERT Pretraining Approach”, 2019. arXiv: 1907.11692. [Online]. Available: <http://arxiv.org/abs/1907.11692>.
- [29] Crispin Almodovar, Fariza Sabrina, Sarvnaz Karimi, and Salahuddin Azad, “Can language models help in system security? investigating log anomaly detection using BERT”, in *Proceedings of the The 20th Annual Workshop of the Australasian Language Technology Association*, Adelaide, Australia: Australasian Language Technology Association, Dec. 2022, pp. 139–147. [Online]. Available: <https://aclanthology.org/2022.alta-1.19>.
- [30] Leslie N. Smith and Nicholay Topin, “Super-convergence: very fast training of neural networks using large learning rates”, 2019, p. 36, ISBN: 9781510626775. DOI: 10.1117/12.2520589. arXiv: 1708.07120.
- [31] Jeremy Howard and Sebastian Ruder, “Universal language model fine-tuning for text classification”, in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Melbourne, Australia: Association for Computational Linguistics, Jul. 2018, pp. 328–339. DOI: 10.18653/v1/P18-1031. [Online]. Available: <https://aclanthology.org/P18-1031>.
- [32] Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, et al., “Don’t Stop Pretraining: Adapt Language Models to Domains and Tasks”, 2020, pp. 8342–8360. DOI: 10.18653/v1/2020.acl-main.740. arXiv: 2004.10964.
- [33] Ananya Kumar, Aditi Raghunathan, Robbie Jones, Tengyu Ma, and Percy Liang, “Fine-Tuning can Distort Pretrained Features and Underperform Out-of-Distribution”, Feb. 2022. arXiv: 2202.10054. [Online]. Available: <http://arxiv.org/abs/2202.10054>.
- [34] Wei Xu, Ling Huang, Armando Fox, David Patterson, and Michael I Jordan, “Detecting large-scale system problems by mining console logs”, in *ICML 2010 - Proceedings, 27th International Conference on Machine Learning*, 2010, pp. 37–44, ISBN: 9781605589077.
- [35] Adam Oliner and Jon Stearley, “What supercomputers say: A study of five system logs”, in *Proceedings of the International Conference on Dependable Systems and Networks*, 2007, pp. 575–584, ISBN: 0769528554. DOI: 10.1109/DSN.2007.103.
- [36] Leland McInnes, John Healy, and James Melville, “Umap: Uniform manifold approximation and projection for dimension reduction”, *arXiv preprint arXiv:1802.03426*, 2018.